

Architecture & Technical Design

Document: Agent Skill Marketplace

Executive Summary

This document defines a deterministic, high-precision architecture for the Adobe University Hackathon 2026 (Round 3). Operating within strict <5 minute and <50 MB constraints, this architecture abandons aesthetic SEO heuristics (such as "keyword density" and arbitrary scoring indexes) in favor of verifiable technical evidence. By leveraging a five-skill architecture—orchestrated through marketplace.json—the system accurately audits Generative Engine Discoverability and Agentic UX engagement. The system utilizes Conditional Rendering to preserve computational budget, Cross-Signal Validation to virtually eliminate false positives, and targeted API queries to resolve RAG knowledge conflicts.

Adobe PS Interpretation & Round 2 Translation

Adobe requires an objective audit of AI discoverability (can models ingest, retrieve, and correctly attribute facts) and on-site engagement (can agents navigate and execute tasks). Translating our Round 2 "Garuda" findings:

- **Visibility** translates to network access (robots.txt), structured ingestion (llms.txt), and semantic extractability (DOM-to-AOM mapping).
- **Entity Confusion** translates to the absence of sameAs Schema.org graphs *coupled with* demonstrable name collisions in global knowledge bases¹.
- **Staleness** translates to internal timestamp contradictions and RAG knowledge conflicts³.
- **Engagement** translates to programmatic accessibility, specifically identifying elements that lack aria-labels and modal traps that blind autonomous agents like WebVoyager⁵.

Design Principles

1. **Observable Evidence Above All:** No finding is reported without pointing to a specific DOM node, network header, or script discrepancy.
2. **Cross-Signal Validation:** "Missing metadata" is an optimization. "Missing metadata + Provable external ambiguity" is a defect.
3. **Strict Budgeting:** Headless browser rendering is conditionally triggered, never default, guaranteeing execution well under 300 seconds.
4. **Mechanism-Sound Remediation:** Fixes must be deterministic, copy-paste developer instructions, not vague marketing advice.

Failure-Mode Taxonomy

Failure Mode	Observable Signal	Detection Method	Confidence	Severity	Recommended Fix
Retrieval	Disallow: /	Deterministic	High	Critical	Remove

Crawler Block	matching OAI-Search Bot or PerplexityBot.	c RFC 9309 parse.			wildcard blocks on retrieval bots; isolate training blocks.
Hydration Blindness	>50% discrepancy between raw HTML text and rendered text.	Conditional text length comparison.	High	High	Implement Server-Side Rendering (SSR) or <noscript> semantic fallbacks.
Agent Modal Trap	Interstitial role="dialog" lacking programmatic close target.	Playwright AOM extraction.	High	Critical	Inject aria-label and role="button" to dismissal overlays.
Entity Collision	Missing sameAs AND >1 Wikidata result for brand name.	Schema extraction + Wikidata API.	High	High	Append definitive Wikidata Q-ID to Organization schema graph.
Contextual Staleness	Schema dateModified heavily contradicts copyright/external date.	Cross-referencing internal vs external timestamps.	Medium	Medium	Sync server-side modification headers with on-page structured data.

Ingestion Friction	Missing /llms.txt.	HTTP 404 on domain root.	High	Low	Publish an llms.txt mapping canonical documentation.
--------------------	--------------------	--------------------------	------	-----	--

Final Skill Architecture (Skill Decomposition)

The architecture uses 5 skills to enforce strict separation of concerns, complying with the agentskills.io standard⁷.

1. **audit-orchestrator (Entrypoint):** Coordinates URL canonicalization, dynamic page selection, skill parallelization, cross-signal fusion, and standardizes the final JSON output.
2. **access-and-curation-auditor:** Evaluates the network layer. It parses robots.txt semantics (recognizing that specific user-agents override wildcards⁹) and checks for the /llms.txt standard¹¹.
3. **semantic-extractability-auditor:** Evaluates the document layer. It calculates the hydration gap (Raw HTML vs Rendered) and flags semantic "div-soup" or critical text trapped in canvas/svg elements that block AI ingestion.
4. **entity-consensus-auditor:** Evaluates the data layer. It extracts Organization and Product JSON-LD, and executes a highly bounded API call to Wikidata to prove actual homonym/entity collisions¹².
5. **agentic-ux-auditor:** Evaluates the interaction layer. It extracts the Accessibility Object Model (AOM) to flag missing interaction labels and inescapable modal traps¹⁴.

End-to-End Execution Flow

Phase 1: Discovery & Page Selection (0–10s)

The orchestrator fetches /sitemap.xml. Using a dynamic regex algorithm, it selects exactly four high-value targets: The Homepage, the newest Article/Blog, a Product/Service page, and the About/Contact page.

Phase 2: Lightweight HTTP Extraction (10–15s)

Fast, concurrent HTTP GET requests pull the raw HTML, robots.txt, and /llms.txt for the selected URLs.

Phase 3: Conditional Render Trigger (15–20s)

The semantic-extractability-auditor compares the word count of the raw <body> against the volume of <script> tags. If the hydration gap indicates a client-side rendered Single Page Application (SPA), it triggers Phase 4. If the site is fully server-rendered, Phase 4 is skipped, saving massive compute time.

Phase 4: Targeted Headless Rendering (20–60s) [Conditional] A sandboxed Playwright session loads the homepage. It intercepts and blocks all media, fonts, and POST requests. It extracts the serialized AOM JSON⁵.

Phase 5: External Verification & Entity Resolution (60–70s)

The entity-consensus-auditor extracts the core brand name. It queries the Wikidata API. If multiple entities share the name and no sameAs canonical link exists on-site, a collision is flagged.

Phase 6: Cross-Signal Fusion & Reporting (70–100s)

The orchestrator merges findings, elevating confidence for overlapping issues, formats deterministic remediation templates, and outputs the final Adobe-compliant JSON.

Detection Algorithms & Strategies

Crawl + Page Selection Strategy

Crawling random URLs wastes budget. We isolate highest-signal pages. The algorithm:

1. GET /sitemap.xml
2. Regex filter for /about/company/ → Target 1 (Entity Context)
3. Regex filter for /product/pricing/ → Target 2 (Structured Data)
4. Regex filter for /blog/news/ sorted by most recent date → Target 3 (Freshness)
5. Domain Root → Target 4 (Agentic UX)

Semantic Extractability Strategy

Generative engines fail when content is semantically unstructured. We utilize BeautifulSoup to parse the HTML5 hierarchy. If the page depth exceeds 15 nodes without utilizing <main>, <article>, or <section>, we flag an Extractability Defect. We explicitly search for text embedded in SVGs or Canvas objects, mapping them as critical accessibility and RAG failures.

Entity & External Web Research Strategy

To verify the "Staleness" and "Entity Confusion" vectors within a 5-minute budget, we use the Wikidata Disambiguation Protocol.

- **Input:** Extracted <title> or Schema Organization name.
- **Action:** 1 API call to [https://www.wikidata.org/w/api.php?action=wbsearchentities&search=\[NAME\]](https://www.wikidata.org/w/api.php?action=wbsearchentities&search=[NAME]).
- **Logic:** If the JSON response contains >1 exact match (e.g., "Garuda (Airline)", "Garuda (Footwear)"), we check the site's JSON-LD. If no sameAs property explicitly links to Q[ID], we report a High-Severity Entity Collision. This replaces slow, hallucinatory LLM web-searches with a 500ms deterministic check.

Agentic UX Strategy

Autonomous agents (e.g., WebVoyager) operate on the AOM, not the visual DOM¹⁵. The Playwright script extracts the AOM tree. The detector logic identifies any node with role="button", role="link", or role="dialog". If an interactive node lacks an aria-label, aria-labelledby, or inner descriptive text, the agent is mathematically blind to it, and a High severity defect is generated¹⁴.

False-Positive & Cross-Signal Validation Strategy

Low-confidence heuristic warnings are completely suppressed. Findings are only elevated to

the report via cross-signal validation:

- **Signal 1:** Missing robots.txt. (Allowed behavior, ignore).
- **Signal 2:** robots.txt blocks GPTBot. (Legitimate IP protection, ignore)¹⁸.
- **Signal 3:** robots.txt blocks OAI-SearchBot. (Critical discovery failure, REPORT)¹⁹. This ensures the evaluator sees zero false-positives regarding crawler logic.

Severity + Confidence Model

- **Critical:** Complete functional block. (e.g., Retrieval bot blocked, unhandled modal trapping AOM).
 - **High:** AI hallucination likely. (e.g., Proven Wikidata entity collision without sameAs).
 - **Medium:** RAG performance degradation. (e.g., Missing metadata timestamps causing knowledge conflicts).
 - **Low:** Optimization. (e.g., Missing /llms.txt).
- Confidence Rule:** Any finding lacking deterministic structural proof (Code snippet, API response) defaults to Low Confidence and is deleted from the final payload.

Evaluation Framework (Synthetic Benchmark)

To ensure generalizability, the system is validated against 4 synthetic archetypes:

1. **Site A (The Wikipedia Standard):** Clean semantic HTML, perfect Schema. *Expected:* 0 findings. Tests False-Positive Rate.
2. **Site B (The SPA Trap):** Client-side rendered React with empty HTML. *Expected:* Hydration Defect triggered. Tests Conditional Rendering recall.
3. **Site C (The Ambiguous Brand):** "Delta" Plumbing with no sameAs. *Expected:* High Entity Collision (Delta Airlines / Delta Faucets). Tests API resolution.
4. **Site D (The Div-Soup Modal):** Newsletter popup using <div> for close button. *Expected:* Critical Agentic UX Trap.

Security Model

The system enforces strict execution boundaries:

- **SSRF Protection:** Python's urllib strictly rejects localhost, 127.0.0.1, and private CIDR ranges.
- **Browser Sandbox:** Playwright launches with --no-sandbox but uses request interception to actively block media, fonts, and script execution originating from unknown third-party domains.
- **Timeout & Rate Limits:** Global 300-second timeout. Hard cap of 10 concurrent requests to the target domain, adhering to polite crawling standards.

Exact Repository Structure

brand-ai-readiness-marketplace/

```
|— marketplace.json
|— README.md
|— skills/
|— audit-orchestrator/
| |— SKILL.md
```

```
| └── scripts/orchestrator.py
| └── access-and-curation-auditor/
|   ├── SKILL.md
|   └── scripts/network_auditor.py
| └── semantic-extractability-auditor/
|   ├── SKILL.md
|   └── scripts/dom_parser.py
| └── entity-consensus-auditor/
|   ├── SKILL.md
|   └── scripts/entity_resolver.py
| └── agentic-ux-auditor/
|   ├── SKILL.md
|   └── scripts/aom_extractor.js
```

marketplace.json Design

JSON

```
{
  "name": "AI Readiness Skill Marketplace"
  "version": "2.0.0"
  "provider": "neutral"
  "entrypoint": "audit-orchestrator"
  "skills": [
    {
      "audit-orchestrator": {
        "path": "./skills/audit-orchestrator"
      }
    },
    {
      "access-and-curation-auditor": {
        "path": "./skills/access-and-curation-auditor"
      }
    },
    {
      "semantic-extractability-auditor": {
        "path": "./skills/semantic-extractability-auditor"
      }
    },
    {
      "entity-consensus-auditor": {
        "path": "./skills/entity-consensus-auditor"
      }
    },
    {
      "agentic-ux-auditor": {
        "path": "./skills/agentic-ux-auditor"
      }
    }
  ]
}
```

Final Report Schema

JSON

```
{
  "site": "https://example.com"
}
```

```

"audited_at" "2026-08-28T02:17:26Z"
"summary" "total_findings" 2 "critical" 1 "high" 1 "medium" 0
"findings"
{
  "finding_id" "UX-01"
  "category" "Agentic UX"
  "title" "AOM Modal Trap"
  "severity" "Critical"
  "confidence" "High"
  "affected_urls" "https://example.com/"
  "evidence" "<div class='popup-close'>X</div> (Missing aria-label and role)"
  "explanation" "Autonomous agents utilizing the Accessibility Object Model cannot perceive this
element to dismiss the modal, resulting in a permanent execution loop."
  "suggested_action" "Modify the element to use semantic HTML: <button aria-label='Close
Dialog'>X</button>."
  "priority" 1
}

```

Works cited

1. Entity Graphs for Generative Engine Optimization - Agenxus, <https://agenxus.com/blog/building-entity-graph-organization-person-schema>
2. Entity Linking with a Knowledge Base: Issues, Techniques, and, https://www.researchgate.net/publication/273397652_Entity_Linking_with_a_Knowledge_Base_Issues_Techniques_and_Solutions
3. Micro-Act: Mitigate Knowledge Conflict in Question Answering via, <https://aclanthology.org/2025.acl-long.909.pdf>
4. ConflictRAG: Detecting and Resolving Knowledge Conflicts ... - arXiv, <https://arxiv.org/html/2605.17301v1>
5. WebVoyager: Autonomous Web Agent Framework - Emergent Mind, <https://www.emergentmind.com/topics/webvoyager>
6. Building an End-to-End Web Agent with Large Multimodal Models, <https://arxiv.org/html/2401.13919v1>
7. What Are Agent Skills and How To Use Them - Strapi, <https://strapi.io/blog/what-are-agent-skills-and-how-to-use-them>
8. Specification - Agent Skills, <https://agentskills.io/specification>
9. Robots.txt Checker — AI Crawler Access | Holp, <https://www.holp.ai/robots-txt-checker/>
10. RFC 9309: Robots.txt Is Now an Official IETF Internet Standard, <https://www.searchengineworld.com/rfc9309-robots-txt-quietly-became-an-official-internet-standard>
11. What Is LLMs.txt? Format, Example, and Where to Place It (2026),

- <https://www.infrasity.com/blog/llms.txt>
12. Entity Clarity and Disambiguation: What They Are and Why They, <https://searchatlas.com/blog/entity-clarity-and-disambiguation/>
 13. How AI models select companies to refer? - Dashboa, <https://dashboa.com/blog/how-ai-selects-companies>
 14. State-of-the-Art Autonomous Web Agents (2024–2025) - Medium, https://medium.com/@learning_37638/state-of-the-art-autonomous-web-agents-2024-2025-3d9d93a5dde2
 15. How AI Agents See Your Website: The Accessibility Tree Explained, <https://isagentready.com/en/blog/how-ai-agents-see-your-website-the-accessibility-tree-explained>
 16. Mind2Web: Towards a Generalist Agent for the Web - GitHub Pages, <https://osu-nlp-group.github.io/Mind2Web/>
 17. Web Scraping With Playwright, <https://scrape.do/blog/web-scraping-with-playwright/>
 18. robots.txt in the age of AI crawlers - Flavio Copes, <https://flaviocopes.com/robots-txt-ai-crawlers/>
 19. Robots.txt for AI Crawlers: GPTBot, ClaudeBot (2026) - WitsCode, <https://witscode.com/blogs/robots-txt-strategy-2026-managing-ai-crawlers>